ChatGPT

Summarize ML Compiler Article

You

Please summarize ths: **https://tvm.apache.org/2026/11/03/rethinking-abstractions-in-the-agentic-era** - and I don't know what a ML compiler is - so I might need some context to understand the discussion.

ChatGPT

The paper is making a fairly provocative argument: **AI coding agents may change what an "ML compiler" is for.** The authors are not saying compilers disappear. They are saying that some of the abstraction layers built to make *human* programmers productive may become less important, while the parts that give programs structure, analyzability, and verifiability may become *more* important. Apache TVM

First: what is an ML compiler?

A conventional compiler takes something like C++ and turns it into machine instructions. An **ML compiler** does something similar for machine-learning computations, especially tensor operations destined for CPUs, GPUs, or accelerators.

Suppose PyTorch conceptually says:

```
C = A × B
```

where these are giant matrices. There are an enormous number of ways to carry out that multiplication on a GPU: how to divide it among GPU threads, how much data to keep in registers or shared memory, how to arrange memory accesses, when to synchronize, and so forth. A good implementation can easily be several times faster than a naive one.

Systems such as TVM, Triton, XLA, MLIR-based stacks, etc. provide languages and compiler machinery that turn a relatively high-level description into highly optimized **GPU kernels**—small programs that actually execute on the GPU.

Historically, one big purpose of the ML compiler has therefore been:

human writes simple description → compiler automatically figures out gnarly GPU details.

Take your curiosity further
Scan to get the ChatGPT app

What changed with agents?

The TVM authors point out that modern coding agents can increasingly do the gnarly part themselves. They can write CUDA, inspect profiler output, modify the code, benchmark it, and repeat dozens or hundreds of times. Some recent agent systems are already producing very high-performance GPU kernels. Apache TVM

That creates an uncomfortable question for compiler researchers:

> If an agent can just write excellent CUDA directly, why do we need a sophisticated compiler abstraction that was designed to protect humans from CUDA?

That is really the intellectual starting point of the article.

Their answer is roughly:

Maybe we don't need as much "compiler knows best" machinery. But we still badly need compiler infrastructure.

Why compiler infrastructure still matters

Their strongest argument is about **analysis and verification**.

Imagine an agent generates some very clever CUDA kernel. It passes 1,000 tests. Is it actually correct for all inputs? Does it ever read outside an array? Does it have a race condition? Is synchronization correct?

You don't want the answer to be:

> "Claude says it looks fine."

And exhaustive testing isn't enough either.

So they argue that you still want a rigorous, structured representation of the program—an **IR**, or *intermediate representation*. Think of an IR as a machine-readable mathematical representation of what the program means, rather than just a blob of CUDA text. Once you have that structure, conventional algorithms can cheaply check memory accesses, synchronization properties, symbolic bounds, types, etc., and more ambitious tools can formally verify correctness. They even mention representing semantics formally in Lean. Apache TVM

Take your curiosity further
Scan to get the ChatGPT app

This is probably the connection most relevant to the AI discussions you've been having recently: **agents generate; deterministic tools check.**

The authors use a nice implicit analogy: an LLM *could* do arithmetic itself, but it is usually smarter to give it a calculator. Likewise an agent could reason line-by-line about every memory access, but a compiler analysis can answer many such questions cheaply, deterministically, and reliably. Apache TVM

Their proposed architecture

They therefore want to shrink the compiler down to something more like a **stable substrate for agents**.

Instead of a giant monolithic compiler containing hundreds of opinionated optimization passes, they propose a small core with a stable AST/IR, standard program-analysis utilities, basic code generation, and easy extension mechanisms. Agents can then build experimental optimization passes, checkers, new operators, or GPU strategies around it without constantly modifying and rebuilding the whole compiler. Apache TVM

Their TIRx project is their attempt to build this. It emphasizes a few things:

- a small, stable representation of programs;
- easy inspection and transformation of that representation;
- dynamic extensions, so agents can add new operations without rebuilding the compiler;
- first-class Python, C++, and Rust interoperability;
- reuse of mature backends such as LLVM and CUDA rather than rebuilding everything themselves. Apache TVM

One particularly revealing detail is that their basic `tirx.compile()` deliberately **doesn't apply clever heuristic optimizations**. It essentially hands the described computation to LLVM. That's almost the opposite philosophy from traditional optimizing compilers: the sophisticated optimization can increasingly live *outside* the stable core and be driven by agents. Apache TVM

The deeper idea

I think the most interesting sentence in the post is effectively this one:

> In the future, we may have compiler infrastructure even when the “compiler” isn't doing much compiling.

Take your curiosity further
Scan to get the ChatGPT app

The compiler becomes less like an oracle that transforms your program and more like a **formal workspace in which agents manipulate programs**.

A rough before-and-after is:

Old model

Human
↓
High-level DSL
↓
Smart compiler with lots of heuristics
↓
CUDA / machine code

Agentic model

Agent ↔ profiler/tests/checkers
↓
Stable program representation (IR)
↓
CUDA / LLVM / existing toolchains

The intelligence moves upward into the agent; the lower layer becomes deliberately boring, predictable, and trustworthy.

Why I think this article is interesting beyond ML compilers

There's a much broader software-engineering thesis hiding in here.

A lot of software abstractions exist partly because **humans can't simultaneously reason about huge numbers of low-level details**. Agents change that constraint. So some abstractions may no longer be optimal.

But abstraction doesn't disappear. Instead, its purpose changes from:

"hide complexity from the programmer"

to

"give the agent a stable interface, structured state, and independently checkable semantics."

That is quite a consequential distinction.

Take your curiosity further
Scan to get the ChatGPT app

It also leads to a useful rule of thumb for agentic scientific software:

Let the agent do the creative search; let conventional software do the cheap deterministic checking; reserve formal methods for the strongest guarantees.

That is very close to what the TVM group is proposing for GPU kernel development. Apache TVM

If you'd like, I can also explain **IR / AST / DSL / CUDA / kernel / LLVM** using one concrete worked example—say, taking a simple `A[i] = B[i] + 1` operation all the way from PyTorch-style code to what runs on a GPU. That would make almost all of the terminology in this article fairly intuitive.

You

Ok - look at the awkward array library - please frame a discussion around awkward given this approach (and I'm thikning not necessarily running on GPU, but also on CPU).

ChatGPT

Yes. Awkward is actually a very interesting test case for this idea, because it already contains something that looks a lot like the "stable substrate" the TVM authors are arguing for.

Awkward's distinctive value is not really "fast C++ kernels." It is that it has a **precise representation of irregular, nested data**—lists of variable length, optional values, records, unions, etc.—together with well-defined operations on those structures. Internally, an array such as

```
[
  [{pt: 20, eta: 1.2}, {pt: 35, eta: -0.4}],
  [],
  [{pt: 50, eta: 0.3}]
]
```

is represented as a composition of layout nodes such as `ListOffsetArray`, `RecordArray`, and primitive `NumpyArray` buffers. Awkward can losslessly describe that structure using its **Form plus buffers** representation. Awkward Array +1

That starts to look remarkably like an IR.

Take your curiosity further
Scan to get the ChatGPT app

The TVM argument translated into Awkward

Today, conceptually, Awkward is something like:

```
User writes high-level Awkward expression

    jets[abs(jets.eta) < 2.5].pt

             ↓

Awkward understands broadcasting,
jagged dimensions, masks, records, etc.

             ↓

sequence of Awkward operations

             ↓

NumPy operations + Awkward C/CUDA kernels

             ↓

CPU / GPU
```

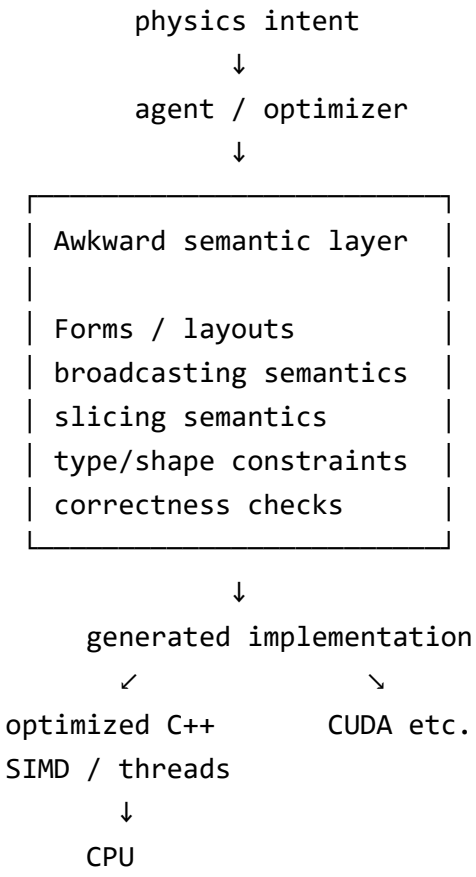
Awkward explicitly has low-level kernels for operations that can't simply be handed to NumPy/CuPy. Those kernels operate on preallocated buffers through a C-like interface, and there are corresponding CPU/GPU implementations. Awkward Array +1

The TVM article suggests asking:

> What if we stop assuming the library/compiler itself must know the optimal implementation of every operation?

Instead, Awkward might become:

Take your curiosity further
Scan to get the ChatGPT app



And crucially, I don't think **GPU is the main point at all**.

CPU is possibly a better example for Awkward.

Consider a HEP analysis

Suppose I write something conceptually like:

```
good_jets = events.jets[
    (events.jets.pt > 30)
    & (abs(events.jets.eta) < 2.4)
]

ht = ak.sum(good_jets.pt, axis=1)
```

Today this is expressed through several Awkward operations. There may be intermediate masks, indexed arrays, broadcasts, reductions, kernel calls, and temporary buffers.

Take your curiosity further
Scan to get the ChatGPT app

A sufficiently capable agent could instead look at:

- the exact Awkward Form;
- the expression being evaluated;
- CPU architecture;
- actual array sizes and jaggedness;
- cache behavior;
- thread count;
- profiling results;

and generate one specialized CPU loop:

```
for each event:
    ht = 0
    for each jet belonging to event:
        if pt > 30 and abs(eta) < 2.4:
            ht += pt
    output[event] = ht
```

Maybe it uses AVX-512. Maybe it parallelizes events with OpenMP. Maybe it chooses not to vectorize because the branches make that slower. Maybe it specializes to 32-bit offsets because this dataset permits them.

The agent can discover that experimentally.

That is almost exactly the TVM argument, transferred from GPU kernels to irregular-array computation.

And here's where Awkward becomes especially interesting

I would distinguish two things that are currently somewhat bundled together in our mental model of Awkward.

Awkward as an execution engine

versus

Awkward as a semantics for irregular arrays.

The agentic argument says the second might ultimately be more important.



Take your curiosity further

Scan to get the ChatGPT app



Awkward's current data model tells you extremely important things:

```
ListOffsetArray
    offsets → [...]
    content →
        RecordArray
            pt  → NumpyArray[float32]
            eta → NumpyArray[float32]
```

That structure tells an agent exactly where the data are and what relationships must hold.

And the Awkward operation:

```
ak.sum(x.pt[x.pt > 30], axis=-1)
```

provides a declarative description of **what answer is wanted**.

Those two pieces together are enough to permit enormous freedom in **how** to calculate it.

That is precisely where an agent becomes useful.

So I might reinterpret Awkward's layers this way

Layer	Role in an agentic world
`ak.Array` API	Human-facing language describing desired analysis
Awkward Types / Forms	**Semantic contract / IR for the data**
Awkward operations	**Semantic specification of transformations**
Typetracer	Static reasoning about structure without materializing data
CPU/CUDA kernels	Reference/trusted implementations
Agent-generated code	Fast specialized implementation
NumPy / LLVM / C++ / CUDA	Stable execution backends



Take your curiosity further

Scan to get the ChatGPT app



The interesting one there is **Typetracer**. Awkward already has a backend that represents arrays without actual data, allowing operations to reason about structure. Awkward Array

That is beginning to look remarkably compiler-ish.

Where I think this gets genuinely provocative

Imagine telling an agent:

> Make this Coffea/Awkward analysis run as fast as possible on this 64-core AMD machine. It must produce results bit-for-bit equivalent to this Awkward expression.

You don't necessarily want the agent to “improve Awkward.”

You could let it generate whatever implementation wins:

```
Awkward expression
       ↓
semantic analysis of Form
       ↓
agent generates candidate implementation
       ↓
compile with clang/LLVM
       ↓
benchmark
       ↓
inspect perf counters
       ↓
rewrite
       ↓
benchmark
       ↓
...
```

It might discover that the best solution is:

Take your curiosity further
Scan to get the ChatGPT app

- one fused C++ loop;
- three loops;
- AVX;
- no AVX;
- TBB threading;
- OpenMP;
- NumPy;
- existing Awkward kernels;
- or some strange hybrid nobody would want to maintain manually.

And that's okay, because **the generated implementation isn't necessarily long-lived source code**.

That parallels one of the important observations in the TVM article: generated kernels can evolve or be discarded rapidly, while the underlying IR and analysis machinery should be stable and carefully maintained. Apache TVM

That would change what we optimize inside Awkward

Traditionally, we might ask:

> How can we make `ak.sum`, slicing, masking, broadcasting, etc. individually faster?

The agentic version of that question is more like:

> Can Awkward expose enough semantics that an agent can safely replace an entire chain of those operations with one implementation?

That's a much bigger opportunity.

For example:

```
jets = events.Jet
selected = jets[(jets.pt > 25) & (abs(jets.eta) < 2.5)]
```

Take your curiosity further
Scan to get the ChatGPT app

```
pairs = ak.combinations(selected, 2)
mass = (pairs["0"] + pairs["1"]).mass
best = ak.argmin(abs(mass - 91.2), axis=1)
```

A conventional library sees roughly:

```
filter
 → construct selected jagged array
 → combinations
 → construct pairs
 → vector addition
 → calculate mass
 → subtraction
 → abs
 → reduction
```

Each abstraction is reasonable for a human.

An agent doesn't have to respect those abstraction boundaries at execution time.

It can understand the whole thing as:

> "For every event, inspect qualifying jet pairs and return the pair whose invariant mass is closest to the Z mass."

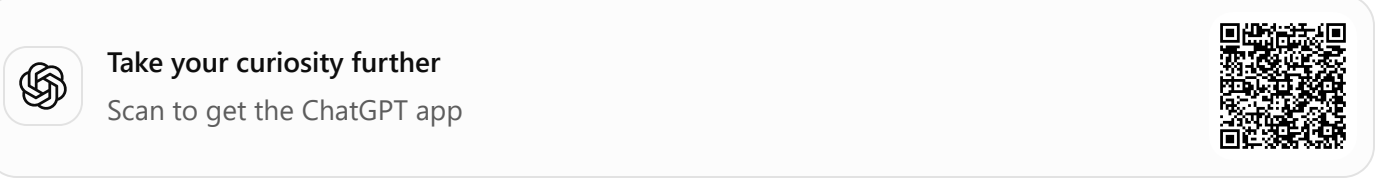
and emit a specialized loop that never constructs most of those intermediate Awkward objects.

That could be a **huge** change.

And it's much more analogous to a database query optimizer than a traditional Python library.

But Awkward provides the crucial safety boundary

This is where I strongly agree with the TVM article's philosophy.



I would *not* want:

```
LLM reads Python
 → writes optimized C++
 → seems to work
```

That's dangerous.

I would want:

```
Awkward semantics
       ↓
precisely defined IR
       ↓
agent produces transformation
       ↓
checker verifies:

  output Form
  indexing validity
  offset bounds
  missing-value semantics
  broadcasting semantics
  reduction semantics
  aliasing / mutation assumptions

       ↓
reference comparison on generated cases
       ↓
optimized executable
```

In other words:

Awkward becomes part specification, part verifier.

That fits almost perfectly with the TVM authors' argument that compiler infrastructure remains necessary even if the agent takes over optimization: structured representations let cheap deterministic analyses catch errors, and correctness needn't depend on trusting the model. Apache TVM

Take your curiosity further
Scan to get the ChatGPT app

There's another important advantage specific to Awkward

Agents are unusually well suited to optimizing Awkward computations because performance is **very data-dependent**.

Imagine two arrays with the same type:

```
1000000 * var * float32
```

One might have average list length 2.

Another might have average list length 500.

Their optimal execution strategies might be completely different.

Likewise:

- almost all lists might be empty;
- offsets might fit in 32 bits;
- optional values might occur 0.001% of the time;
- one union branch might account for 99.99% of values;
- event sizes might make threading worthwhile or disastrous;
- fields might or might not fit comfortably in cache.

A traditional library must choose a fairly general algorithm.

An agent can say:

> For **this dataset, this analysis, and this CPU**, what is fastest?

Then profile it and find out.

That is exactly the kind of heuristic optimization TVM is questioning whether humans should encode permanently into giant compiler passes. Apache TVM



Take your curiosity further

Scan to get the ChatGPT app



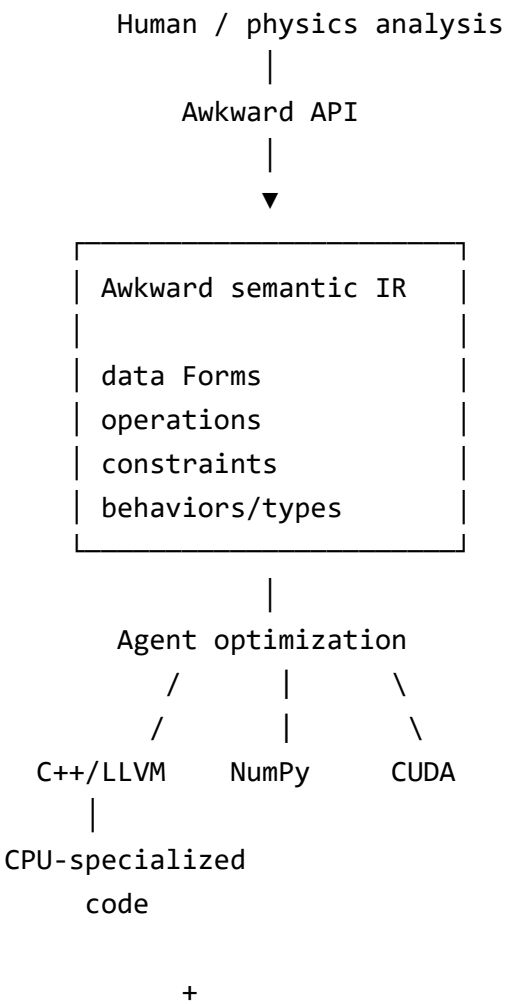
I therefore wouldn't describe the future as “compile Awkward”

I'd describe it as:

Awkward becomes the semantic substrate for irregular data computation.

And above it you have agents that can generate execution strategies.

Something like:



Take your curiosity further
Scan to get the ChatGPT app

```
independent
  checker
```

And interestingly, **Awkward already possesses a surprising fraction of the bottom-middle box**: layouts, Forms, buffers, backends, typetracer, and low-level kernel specifications. Its arrays are explicitly designed as tree-like structures backed by contiguous buffers, and the kernel interface already separates low-level manipulation from the higher-level representation. Awkward Array +1

What it doesn't really have yet is a first-class representation of an **entire computation** as something an optimizer can freely rewrite.

That, to me, is the missing piece.

And that leads to what I think is the most interesting question for Awkward:

Should Awkward's next major abstraction be an operation IR—not another faster kernel implementation?

That IR could represent a whole Awkward computation graph while retaining exact jagged-array semantics. Then an agent could fuse, specialize, reorder, parallelize, and code-generate against it while Awkward remains the authority on what the computation *means*.

If you take the TVM article seriously, that could ultimately be considerably more valuable than spending a lot of effort hand-optimizing another generation of individual Awkward CPU kernels.

You

I wonder if one could have a python annotaion (like @agent_optimiz) in front of a def xxx method, and then that method contains, say, awkward calls, and then does an optimization like ths. So a "compiler" could come along and build a "rust" (hahahaha) implementation that would know awkward array...

ChatGPT

Yes — I think that is a very natural way to expose it to a physicist/user. And the interesting part is that the **decorator is not really asking Python to compile the function in the traditional sense**. It is saying:

Take your curiosity further
Scan to get the ChatGPT app

> “This function is a semantic specification. You are allowed to replace its implementation, provided you preserve its Awkward semantics.”

Something like:

```
@agent_optimize
def select_jets(events):
    jets = events.Jet
    good = jets[(jets.pt > 30) & (abs(jets.eta) < 2.4)]
    return ak.sum(good.pt, axis=1)
```

The first time this gets used, the system could inspect the function and the concrete Awkward `Form` of `events`, observe that this is essentially a filter followed by a reduction, and then generate a specialized implementation.

And yes, **Rust would actually be a very plausible target**—joking aside.

Awkward already has a somewhat similar precedent: it extends Numba so that JIT-compiled functions can understand Awkward Arrays. But today's Numba interface is much more restrictive: inside compiled code you largely iterate through Awkward data rather than freely writing ordinary chains of `ak.*` operations. Awkward Array +1 What you're proposing is substantially higher level.

What I think the decorator should mean

I wouldn't have this:

```
@agent_optimize
```

simply ask an LLM:

> Rewrite the following Python in Rust.

Instead I'd have several stages:

```
Python function
   ↓
capture Awkward operations
```



Take your curiosity further

Scan to get the ChatGPT app



```
   ↓
Awkward computation graph / semantic IR
   ↓
specialize to input Form
   ↓
agent proposes implementation
   ↓
Rust / C++ / LLVM / whatever
   ↓
compile
   ↓
validate against reference Awkward implementation
   ↓
cache optimized implementation
```

So if you write:

```
@agent_optimize
def ht(events):
    jets = events.Jet
    jets = jets[jets.pt > 30]
    return ak.sum(jets.pt, axis=1)
```

the internal representation might become something like:

```
input:
    ListOffsetArray[
        Record {
            pt: float32,
            eta: float32,
            ...
        }
    ]

operation:
    per event:
        select records where pt > 30
        extract pt
```

Take your curiosity further
Scan to get the ChatGPT app

```
        sum pt

output:
    float32[event]
```

Then the agent is free to notice:

There is absolutely no reason to construct the filtered `jets` array.

It can produce essentially:

```
for event in events {
    let mut sum = 0.0;

    for jet in event.jets {
        if jet.pt > 30.0 {
            sum += jet.pt;
        }
    }

    output.push(sum);
}
```

except operating directly on the Awkward buffers and offsets rather than constructing Rust objects.

That could eliminate potentially several intermediate arrays and memory passes.

Why Rust is particularly amusingly appropriate

The target language actually matters less than one might initially think.

The important interface would be something like:

```
Awkward Form
+
raw buffers
+
well-defined Awkward semantics
```

Awkward already separates its logical nested representation from contiguous underlying buffers, and its lower-level operations ultimately work against those buffers. Awkward Array

Take your curiosity further
Scan to get the ChatGPT app

So generated Rust could receive something analogous to:

```
event_offsets: &[i64]
jet_pt:        &[f32]
jet_eta:       &[f32]
```

and emit:

```
ht: Vec<f32>
```

It doesn't need an elaborate Rust-native implementation of `ak.Array`.

It only needs to understand the **contract of an Awkward Form**.

That is a big distinction.

You wouldn't necessarily build:

> "Awkward Array for Rust."

You'd build:

> **a tiny Rust runtime capable of consuming and producing Awkward buffers according to a Form.**

That is much easier.

And it could be aggressively specialized

Suppose the actual incoming array is:

```
10000000 * {
    Jet: var * {
        pt: float32,
        eta: float32,
        phi: float32,
        mass: float32
    }
}
```



Take your curiosity further

Scan to get the ChatGPT app



The compiled routine might be keyed on something like:

```
function source/hash
+
input Form
+
CPU architecture
+
optimization policy
```

So:

```
ht(events)
```

might transparently do:

```
Do I already have an implementation for:

    ht
    this Awkward Form
    x86-64 AVX2
    ?

yes → call compiled implementation

no  → run reference Awkward version
      generate candidates
      verify
      benchmark
      cache winner
```

Eventually you could even have multiple versions:

```
ht / Form A / generic x86
ht / Form A / AVX2
ht / Form A / AVX-512
ht / Form B / AVX2
```

It's basically JIT specialization, except **the optimizer itself is an agent** rather than a fixed compiler pass.

Take your curiosity further
Scan to get the ChatGPT app

There's a beautiful property of using a Python decorator

It provides an extremely clean boundary between **physics code** and **implementation code**.

The physicist continues writing:

```
@agent_optimize
def dilepton_mass(events):
    leptons = events.Lepton[events.Lepton.pt > 25]
    pairs = ak.combinations(leptons, 2)
    pair = pairs[
        (pairs["0"].charge * pairs["1"].charge) < 0
    ]
    return (pair["0"] + pair["1"]).mass
```

That remains readable, inspectable, runnable Awkward code.

It is also the **reference implementation**.

You don't replace it.

The generated implementation is just an optimization cache.

That's important because six months later someone can still read the analysis and understand what it means.

The Rust could be horrific:

`generated/83fac91b97e2.rs`

and nobody cares.

Delete the cache, and the Python still works.

That is a fundamentally different model from today's usual:

> Rewrite the performance-critical part in C++ and now maintain two implementations forever.



Take your curiosity further

Scan to get the ChatGPT app



The verification story becomes unusually strong

This is the thing I like most about this idea.

Because the decorated Python remains executable, the optimizer always has an oracle:

```
expected = original_python(test_data)
actual   = optimized_rust(test_data)

assert awkward_semantically_equal(expected, actual)
```

And you can generate evil Awkward inputs automatically:

- empty events;
- empty sublists;
- missing values;
- unusual offsets;
- length-1 lists;
- enormous lists;
- NaNs;
- infinities;
- union types;
- masked values;
- boundary conditions.

Then run property-based tests.

Even better, some things don't require runtime testing at all. From the Form and operation IR you could statically check that:

```
output shape is right
offset relationships are valid
index accesses are bounded
null semantics are maintained
input buffers aren't mutated
```

So the trust hierarchy becomes:

```
Human writes readable Awkward
          ↓
```

Take your curiosity further
Scan to get the ChatGPT app

```
Awkward defines semantics
          ↓
Agent creates fast implementation
          ↓
deterministic tools verify structure
          ↓
differential tests verify results
          ↓
compiler produces executable
```

That's very much the philosophy of the TVM article.

There's one thing I would change in your syntax

I might make the distinction explicit between **compile now** and **this function is permitted to be optimized**.

Perhaps:

```
@ak.optimize
def ht(events):
    ...
```

means:

> This function defines a pure Awkward computation and may have alternative implementations.

Then:

```
ht.optimize(events)
```

could deliberately initiate an optimization search, while ordinary:

```
ht(events)
```

runs either the cached optimized version or the reference implementation.

Or perhaps:



Take your curiosity further

Scan to get the ChatGPT app



```
@ak.compile(optimizer="agent")
def ht(events):
    ...
```

although I actually like `optimize` more than `compile`, because what's happening is more profound than normal compilation.

There is one major architectural prerequisite

This gets back to the previous answer.

Awkward would need a clean way to turn:

```
def f(x):
    y = x[x.pt > 30]
    return ak.sum(y.pt, axis=1)
```

into a **whole-function Awkward computation graph**.

Right now Awkward knows how each individual operation works. It doesn't naturally hold the whole chain as a durable IR.

You'd want something resembling:

```
%0 = field %events "Jet"
%1 = field %0 "pt"
%2 = greater %1 30
%3 = mask %0 %2
%4 = field %3 "pt"
%5 = reduce_sum %4 axis=-1
return %5
```

with Awkward's jagged semantics attached to every operation.

Once you have *that*, the agent optimization part is almost separable from Awkward itself.

And this is where something like Typetracer could potentially be useful: Awkward already has machinery for propagating structural information without needing concrete array values, so it already has part of the infrastructure needed to reason about computations symbolically. Its current JIT ecosystem also

Take your curiosity further
Scan to get the ChatGPT app

demonstrates that Awkward layouts can already participate in compiled execution. Awkward Array

In fact, I think this could be broader than Awkward

The really neat version might allow:

```
@agent_optimize
def analysis(events):
    jets = events.Jet
    electrons = events.Electron

    good_jets = jets[jets.pt > 30]

    ht = ak.sum(good_jets.pt, axis=1)
    njets = ak.num(good_jets)

    return ak.zip({
        "ht": ht,
        "njets": njets,
    })
```

and optimize the **entire analysis region**.

Now you're no longer talking about making one `ak.sum` fast.

You're letting the optimizer discover:

> "I can compute `ht` and `njets` in the same pass over `Jet_pt`, and I never have to materialize `good_jets` at all."

On a CPU, that kind of **fusion and elimination of memory traffic** could easily be much more valuable than clever instruction-level optimization.

And because HEP analyses tend to consist of long chains of exactly these transformations, I suspect that's where the really large gains would be.

Take your curiosity further
Scan to get the ChatGPT app

So yes: I think your `@agent_optimize` idea is more than syntactic sugar. **It gives a very clean programming model for exactly the architectural shift that the TVM article is suggesting**: the Python/Awkward function defines meaning; the agent owns implementation; Awkward supplies the structured data semantics and the correctness oracle; Rust/C++/LLVM is just disposable machinery underneath.

Take your curiosity further
Scan to get the ChatGPT app